

Roteiro técnico de refatoração do Fantasy NBA System

Este documento consolida o plano técnico de evolução da aplicação, com foco em estabilização arquitetural, melhoria de manutenção e refatoração progressiva das páginas existentes. A estrutura atual já possui base funcional consistente, com autenticação centralizada no shell da aplicação, páginas separadas por domínio e forte uso de serviços auxiliares, mas ainda apresenta repetição de helpers, acoplamento entre UI e regra de negócio e maturidade desigual entre os módulos.[file:679][file:680][file:681][file:682][file:683]

A estratégia proposta é executar a evolução em duas camadas. Primeiro, estabilizar os blocos transversais — autenticação, sessão, permissões, helpers visuais e padronização estrutural — e depois refatorar cada página sobre essa base comum. A prioridade recomendada é começar por `uni.py` e helpers compartilhados, seguir para `teams.py`, depois `classificacao.py`, `home.py` e `lei.py`. [file:679][file:680][file:681][file:682][file:683]

Arquitetura geral

Hoje a aplicação está organizada em torno de um shell principal que cuida de sessão, login, bootstrap e navegação, enquanto as páginas específicas assumem responsabilidades próprias de visualização e operação. Essa base é boa para continuidade, mas a repetição de funções como `require_login_v5()` e verificações de admin em vários arquivos indica que parte da infraestrutura transversal ainda está distribuída de forma excessiva.[file:679][file:680][file:681][file:682][file:683]

O objetivo da arquitetura proposta é separar claramente quatro camadas: shell da aplicação, helpers transversais, serviços de domínio e páginas de interface. Com isso, cada arquivo deixa de acumular múltiplas funções e o sistema passa a ter pontos mais claros para manutenção, testes e futuras expansões.[file:679][file:680][file:681][file:682][file:683]

Princípios da refatoração

- Centralizar autenticação, sessão e papéis de usuário em helpers compartilhados.[file:679][file:680][file:681][file:682][file:683]
- Reduzir lógica de negócio dentro dos arquivos de página, movendo cálculos, normalizações e validações para services.[file:680][file:682][file:683]
- Padronizar estados visuais recorrentes, como mensagens de vazio, avisos, cards de métricas e captions de contexto.[file:681][file:682][file:683]
- Refatorar primeiro os pontos de maior risco operacional, especialmente o fluxo de transactions em `teams.py`. [file:683]

Plano por arquivo

uni.py

`uni.py` hoje concentra configuração da aplicação, healthcheck do banco, inicialização de schema, fechamento de bids expiradas, gestão de sessão, login, troca obrigatória de senha e navegação entre páginas. Isso o torna funcional como entrypoint, mas ainda sobrecarregado com responsabilidades que são reutilizadas pelo restante do app.[file:679]

Objetivo

Transformar `uni.py` em um shell enxuto, responsável apenas por inicialização, roteamento e composição da estrutura principal da aplicação. Tudo que for reutilizável fora do entrypoint deve ser deslocado para helpers próprios.[file:679]

O que extrair

- `init_session_v5()` para um helper de sessão.[file:679]
- `logout_v5()` para um helper de sessão/autenticação.[file:679]
- `render_login_v5()` para um módulo de UI de autenticação.[file:679]
- `render_force_password_change_v5()` para um módulo de UI de autenticação/perfil.[file:679]
- `get_environment_label_v5()` e `startup_v5()` para um módulo de bootstrap da aplicação.[file:679]

Estrutura sugerida

Novo módulo	Responsabilidade	Origem inicial
<code>app_lib/session_helpers.py</code>	usuário logado, inicialização de sessão, logout, login obrigatório	<code>uni.py</code> e páginas[file:679][file:680][file:681][file:682][file:683]
<code>app_lib/auth_ui.py</code>	renderização de login e troca obrigatória de senha	<code>uni.py</code> [file:679]
<code>app_lib/app_boot.py</code>	startup, healthcheck, label de ambiente	<code>uni.py</code> [file:679]
<code>app_lib/role_helpers.py</code>	verificação de admin e guards simples	<code>home.py</code> , <code>teams.py</code> [file:681][file:683]

Ajustes técnicos

- Avaliar o custo de `close_expired_bids_v5()` no boot; se estiver pesado, mover para rotina controlada ou condição temporal simples para evitar trabalho desnecessário a cada carregamento.[file:679]
- Criar um renderer de sidebar comum para evitar crescimento do shell ao adicionar novas informações de usuário ou ambiente.[file:679]

- Declarar a navegação em uma estrutura centralizada, facilitando manutenção de páginas futuras.[file:679]

Resultado esperado

`uni.py` passa a ser um entrypoint pequeno, previsível e muito estável. Isso reduz risco de regressão ao evoluir autenticação, perfil ou novas páginas do sistema.[file:679]

`home.py`.

`home.py` hoje funciona como hub de abas dinâmicas, contendo regras, calendário, draft, links, jogos e um mural com posts e comentários. A funcionalidade existe, mas a página ainda atua mais como agregador de blocos do que como uma home realmente orientada à operação da liga.[file:681]

Objetivo

Transformar a Home em dashboard principal da liga, preservando o conceito de abas, mas com hierarquia melhor entre destaques, conteúdo editorial e áreas operacionais.[file:681]

Problemas atuais

- Posts e comentários entram em praticamente qualquer aba por padrão, o que dilui a função de cada área.[file:681]
- Conteúdos importantes, como regra ativa, calendário e draft, não aparecem com destaque de “primeira leitura”. [file:681]
- Helpers como `get_user_display()` e `is_admin_user()` estão locais, quando deveriam ser compartilhados.[file:681]

Refatoração proposta

Separar a página em renderizadores menores:

- `render_home_overview()` para blocos de destaque da liga.[file:681]
- `render_rule_tab()` para regra ativa e histórico futuro, se necessário.[file:681]
- `render_calendar_tab()` para eventos e próximos marcos.[file:681]
- `render_draft_tab()` para board do draft.[file:681]
- `render_links_tab()` para links e jogos.[file:681]
- `render_feed_tab()` para mural, posts e comentários.[file:681]

Melhorias de produto

- Adicionar topo com cards de destaque: regra vigente, próximos eventos, situação do draft e atividade recente.[file:681]
- Tornar posts e comentários uma área específica, em vez de apêndice automático de todas as abas.[file:681]

- Criar padrão visual para “nada cadastrado”, em vez de mensagens dispersas com estilos diferentes.[file:681]

Ajustes técnicos

- Migrar publicação de posts e comentários para `st.form`, evitando comportamento confuso de rerender com `text_area` e `button` soltos.[file:681]
- Reaproveitar helper compartilhado de admin em vez de manter `is_admin_user()` local.[file:681][file:683]
- Manter `ensure_default_home_tabs()` e leitura dinâmica das tabs, mas desacoplar o render do loop principal.[file:681]

Resultado esperado

A Home passa a ser uma tela inicial realmente útil, com leitura executiva no topo e aprofundamento por blocos, sem perder a flexibilidade atual de conteúdo dinâmico.[file:681]

classificacao.py

`classificacao.py` já possui uma base analítica relevante: transforma os jogos em formato longo, calcula vitórias e derrotas, aplica desempate por confronto direto quando há empate entre dois times e monta a matriz de head-to-head. O ponto fraco não está na intenção da regra, mas no acoplamento da lógica à página e na simplicidade visual da entrega final.[file:680]

Objetivo

Criar um módulo de classificação mais robusto, com serviço estatístico separado da interface e uma página orientada à leitura competitiva da temporada.[file:680]

O que mover para service

Todo o bloco abaixo deveria sair da página e ir para algo como `app_lib/standings_service.py`:

- `build_team_lookup()`. [file:680]
- `build_games_long()`. [file:680]
- `build_standings()`. [file:680]
- `head_to_head_record()`. [file:680]
- `apply_head_to_head_tiebreak()`. [file:680]
- `build_head_to_head_matrix()`. [file:680]
- `validate_schedule()`. [file:680]

Limitações atuais a corrigir

- Empates com três ou mais times hoje acabam ordenados alfabeticamente, o que é um critério fraco para desempate final.[file:680]
- `schedule_check_df` é calculado, mas não é exibido ao usuário.[file:680]
- A tabela final mostra apenas vitórias e derrotas, deixando de fora métricas úteis como jogos, aproveitamento, pontos pró e saldo.[file:680]

Nova estrutura de página

Sugerem-se três tabs internas:

Aba	Conteúdo
Classificação	ranking principal, posição, recorde e métricas derivadas.[file:680]
Confronto direto	matriz head-to-head e filtro por time.[file:680]
Calendário	validação de pares, quantidade de jogos por confronto e eventuais inconsistências.[file:680]

Resultado esperado

A página deixa de ser apenas uma exibição simples de duas tabelas e passa a funcionar como módulo de standings, com lógica mais reaproveitável e leitura mais rica para a liga.[file:680]

lei.py

`lei.py` já entrega um conjunto grande de funcionalidades: listagem de jogadores e propostas, histórico por atleta, submissão de lance, visão de cap, administração de propostas, administração de usuários, auditoria e troca de senha. O desafio não é funcionalidade insuficiente, e sim densidade excessiva de responsabilidade num único arquivo.[file:682]

Objetivo

Modularizar a página do leilão sem alterar a regra atual do sistema, reduzindo acoplamento e melhorando legibilidade do código e da experiência do usuário.[file:682]

Refatoração sugerida

Separar as abas em funções dedicadas:

- `render_auction_players_tab()`. [file:682]
- `render_auction_bid_form_tab()`. [file:682]
- `render_auction_cap_tab()`. [file:682]
- `render_auction_admin_tab()`. [file:682]
- `render_auction_profile_tab()`. [file:682]

Além disso, mover helpers de formatação para módulo próprio:

- `valor_por_extenso_v5()`.[\[file:682\]](#)
- `formatar_brl_v5()`.[\[file:682\]](#)
- `format_remaining_v5()`.[\[file:682\]](#)

Melhorias de interface

- Na aba de propostas, adicionar filtros extras além de posição, como status e time.[\[file:682\]](#)
- Na aba de nova proposta, destacar melhor o contexto do jogador selecionado, especialmente proposta ativa atual e tempo restante.[\[file:682\]](#)
- Na aba admin, dividir em subtabs internas: propostas, usuários e auditoria.[\[file:682\]](#)
- Na aba cap, substituir tabela crua por métricas e ordenação mais amigável antes do grid detalhado.[\[file:682\]](#)

Ajustes técnicos

- Reduzir `engine.begin()` espalhado pela página quando consultas puderem vir de services específicos.[\[file:682\]](#)
- Centralizar checks de admin em helper compartilhado.[\[file:682\]](#)[\[file:683\]](#)
- Revisar padrões de formulário para consistência com o restante da aplicação.[\[file:682\]](#)

Resultado esperado

`lei.py` continua sendo um módulo forte em operação, porém com manutenção mais segura e crescimento futuro menos arriscado.[\[file:682\]](#)

teams.py

`teams.py` é o arquivo mais crítico do sistema hoje. Ele já concentra seleção de time, recorte de temporadas, leitura de MAIN e DEV, cálculos de cap, picks, histórico de transactions e criação administrativa de movimentações. É também o ponto com maior densidade de regra de negócio e maior acoplamento entre interface, validação e persistência.[\[file:683\]](#)

Objetivo

Separar `teams.py` em três blocos claros: contexto de dados do time, renderização das tabs e motor de transactions por tipo.[\[file:683\]](#)

Refatoração macro

Extrair builders e renderizadores como:

- `build_team_page_context(...)`.[\[file:683\]](#)
- `render_team_header_summary(...)`.[\[file:683\]](#)
- `render_main_roster_tab(...)`.[\[file:683\]](#)
- `render_dev_roster_tab(...)`.[\[file:683\]](#)

- `render_team_picks_tab(...)`.[\[file:683\]](#)
- `render_transactions_history_tab(...)`.[\[file:683\]](#)
- `render_transaction_admin_form(...)`.[\[file:683\]](#)

Plano específico para transactions

O bloco de transactions deve ser tratado como mini-módulo próprio, porque hoje ainda carrega herança de uma modelagem excessivamente bilateral, mesmo quando a movimentação não é TRADE.[\[file:683\]](#)

Tipos canônicos

Padronizar como base única:

- TRADE.[\[file:683\]](#)
- WAIVE.[\[file:683\]](#)
- ADD.[\[file:683\]](#)
- MOVE.[\[file:683\]](#)
- OTHER.[\[file:683\]](#)

Nova lógica por tipo

Tipo	Comportamento esperado
TRADE	dois lados, times obrigatórios e diferentes, players e picks permitidos. [file:683]
WAIVE	um lado só, sem <code>to_team_id</code> , normalmente removendo player de um time. [file:683]
ADD	um lado de entrada, sem <code>from_team_id</code> , exigindo suporte real no update do roster. [file:683]
MOVE	mesmo time, player obrigatório e troca entre MAIN e DEV. [file:683]
OTHER	fluxo genérico controlado, com validação mínima e sem presumir bilateralidade. [file:683]

Helpers necessários

- `normalize_transaction_context(tx_type, from_team_id, to_team_id)`.[\[file:683\]](#)
- `build_transaction_row(...)`.[\[file:683\]](#)
- `build_transaction_items_rows(...)`.[\[file:683\]](#)
- `validate_transaction_items(...)` como dispatcher por tipo.[\[file:683\]](#)
- builders de UI por tipo, em módulo como `app_lib/transactions_ui.py`.[\[file:683\]](#)

Problemas atuais a tratar

- O fluxo ainda carrega a semântica fixa de “Time A” e “Time B”, que não serve bem para WAIVE, ADD e MOVE.[file:683]
- Há alta duplicação entre o bloco de assets do lado A e do lado B.[file:683]
- ADD ainda depende da forma como `update_rosters()` vai inserir efetivamente um jogador novo no destino, não apenas movê-lo de um source já existente.[file:683]
- Há importações aparentemente não utilizadas, como `require_admin_page` e `admin_only_action`, o que sugere limpeza pendente da página.[file:683]

Melhorias de interface

- Tornar o cabeçalho do time mais semântico, com melhor destaque de cap, red flags e status geral do elenco.[file:683]
- Dar nomes contextuais aos blocos do formulário conforme o tipo da transaction, em vez de manter “Time A” e “Time B” como rótulo universal.[file:683]
- Enriquecer o histórico com filtros adicionais por item ou janela temporal, além de tipo, status e season.[file:683]

Resultado esperado

`teams.py` se torna o módulo mais sólido do sistema, com transações previsíveis, validação coerente por tipo e manutenção muito menos arriscada.[file:683]

Helpers compartilhados

A refatoração só será consistente se alguns módulos transversais forem criados desde o início. Sem isso, o sistema apenas deslocará duplicações de lugar, sem resolver a raiz do problema. [file:679][file:680][file:681][file:682][file:683]

Helpers propostos

Módulo	Escopo
<code>app_lib/session_helpers.py</code>	login obrigatório, usuário atual, sessão e logout.[file:679][file:680][file:681][file:682][file:683]
<code>app_lib/role_helpers.py</code>	<code>is_admin_user</code> , guards e validações simples de papel.[file:681][file:683]
<code>app_lib/page_ui.py</code>	títulos, captions, cards, mensagens de vazio, alerts e padrões de layout. [file:681][file:682][file:683]
<code>app_lib/standings_service.py</code>	toda a lógica de classificação.[file:680]
<code>app_lib/transactions_ui.py</code>	construção e validação da UI de transactions.[file:683]
<code>app_lib/auction_ui_helpers.py</code>	formatação monetária, tempo restante e blocos auxiliares do leilão.[file:682]

Ordem de execução proposta

A execução recomendada segue a lógica de estabilizar primeiro a base comum, depois atacar o módulo mais crítico, e só então elevar as demais páginas para um padrão homogêneo.[file:679][file:680][file:681][file:682][file:683]

Fase	Escopo	Entrega principal
1	uni.py + helpers comuns	centralização de auth, sessão, role e shell da aplicação.[file:679][file:681][file:683]
2	teams.py	refatoração do módulo de transactions e reorganização da página de elencos.[file:683]
3	classificacao.py	extração do standings service e nova estrutura de leitura da classificação.[file:680]
4	home.py	transformação da Home em dashboard de liga com melhor hierarquia.[file:681]
5	lei.py	modularização das abas e limpeza da área administrativa do leilão.[file:682]

Primeiros passos práticos

Como o avanço começará por uni.py e helpers, o primeiro pacote de trabalho deve focar em preparar a fundação que será reaproveitada pelas páginas seguintes. Isso reduz retrabalho ao entrar em teams.py e evita que a refatoração do módulo de transactions dependa de infra provisória.[file:679][file:683]

Sprint 1

- Criar session_helpers.py com get_current_user(), require_login(), init_session() e logout().[file:679][file:680][file:681][file:682][file:683]
- Criar role_helpers.py com is_admin_user() e helpers de acesso.[file:681][file:683]
- Criar auth_ui.py com render de login e troca obrigatória de senha.[file:679]
- Refatorar uni.py para usar apenas esses módulos e manter o shell limpo.[file:679]

Sprint 2

- Entrar em teams.py e isolar contexto, render e transactions.[file:683]
- Construir transactions_ui.py com modelagem por tipo.[file:683]
- Reduzir duplicação entre lados da transaction e consolidar normalização/validação.[file:683]
- Revisar persistência de ADD, WAIVE, MOVE e TRADE no fluxo completo.[file:683]

Encaminhamento recomendado

A ordem proposta é tecnicamente adequada porque começa pelo que sustenta todas as páginas e em seguida avança para o módulo mais sensível da aplicação. Essa sequência reduz risco, melhora consistência e cria uma base sólida para a modernização do restante do sistema.[file:679][file:683][file:680][file:681][file:682]

